LEADING UNIVERSITY

Final Project

Code:CSE – 4116

Web Technologies

Submitted to:

Somapika Das

Lecturer, Department of CSE

Leading University, Sylhet

Submitted by:

Name: Rabiul Islam Apu

ID: 0182220012101**096**

Section: C

Batch: 60th

Submission Date: 20 September 2025

Project Report

Title

Build a Responsive Web Application with User Authentication and Profile Management

Abstract

This report documents the design, implementation, and evaluation of a responsive web application that provides user registration, authentication, and profile management. The system is developed using HTML, CSS, Bootstrap, JavaScript for the front end; PHP for server-side processing; and MySQL for persistent storage. Email communication (verification and password recovery) is implemented via PHPMailer. The application supports full CRUD operations on user profiles, enforces front-end and back-end validation, manages secure sessions, and includes additional features: password reset mechanism, login history (activity log), and a dark/light mode toggle. The report describes system requirements, architecture and database design, implementation details, security measures, testing, deployment instructions, and suggestions for future enhancement.

Keywords

Authentication, PHP, MySQL, PHPMailer, Bootstrap, Responsive Design, CRUD, Session Management, Security

Table of Contents

- Project Management and Documentation
- Limitations and Future Work
- Conclusion
- References
- Appendices

1. Introduction

1.1 Background

User authentication and profile management are foundational components of modern web applications. Ensuring secure and user-friendly registration and login flows, combined with responsive user interfaces, is essential for applications that serve both desktop and mobile users.

1.2 Purpose of the Project

The purpose of this project is to demonstrate the design and development of a fully functional web application that enables users to register, verify their email, log in, and manage their profiles. The system must implement robust validation, secure authentication and session management, and persistent storage of user data.

1.3 Scope

This project covers front-end design (responsive UI), back-end processing (PHP), database design (MySQL), email integration (PHPMailer), and supportive features such as password reset, login history, and a dark/light mode. The implementation emphasizes security best practices relevant to an academic project context.

2. Objectives

The primary objectives are:

- Design responsive registration and login forms.
- Implement front-end (JavaScript + Bootstrap) and back-end (PHP) validation.
- Create a secure authentication mechanism using PHP sessions.
- Implement CRUD operations for user profiles.
- Integrate email verification and password reset functionality using PHPMailer.
- Provide additional features: login history logging and user-selectable dark/light mode.

3. System Analysis

3.1 Functional Requirements

- User Registration with validation and email verification.
- Email-based account activation.
- User Login with secure authentication and session management.
- Profile Create, Read, Update, and Delete (CRUD) operations.
- Password reset via email with time-limited tokens.
- Logging of user login activity (timestamp and IP).
- Responsive interface for desktop and mobile.
- Dark/Light mode toggle persisted per user.

3.2 Non-Functional Requirements

- Security: protect against SQL injection, XSS, CSRF, and session fixation.
- Usability: simple and accessible UI using Bootstrap.
- Maintainability: modular code and clear documentation.
- Performance: efficient database queries and server-side validation.
- Portability: deployable on a standard LAMP (Linux, Apache, MySQL, PHP) stack.

3.3 Use Case Overview

- New User registers -> receives verification email -> verifies account -> logs in -> manages profile.
- Registered User logs in -> views and edits profile -> optionally resets password.
- System records login events for history.

4. System Design

4.1 High-Level Architecture

The application follows a classical client-server architecture:

- Client: HTML/CSS/Bootstrap, JavaScript. Responsible for UI rendering, client-side validation, and AJAX calls where applicable.
- Server: PHP scripts handle routing, server-side validation, authentication, session management, and CRUD operations.
- Database: MySQL stores user records, verification tokens, password reset tokens, and activity logs.
- Mail Service: PHPMailer configured to use SMTP for sending verification and reset emails.

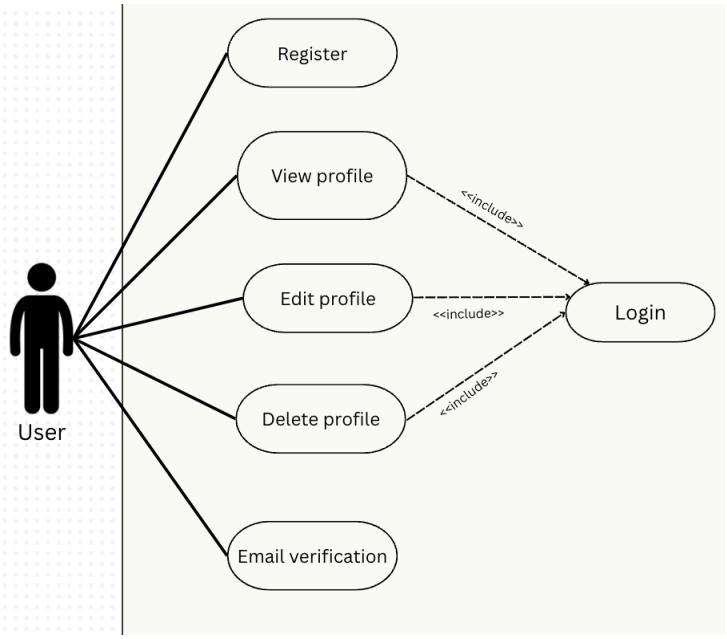
A simplified diagram: **Client (Browser) ⇄ Web Server (Apache + PHP) ⇄ MySQL Database**

4.2 Database Design

4.2.1 Major Tables

- **users**
- **user_profiles**
- **email_verifications**
- **password_resets**

4.2.2 Entity-Relationship Use case Diagram

- 

4.3 Security Design

- Passwords are hashed using password_hash() with PASSWORD_BCRYPT or PASSWORD_ARGON2I (where available).
- All database queries use prepared statements (PDO or mysqli with bound parameters) to prevent SQL injection.
- CSRF tokens are implemented for all state-changing requests.
- Session fixation is mitigated by calling session_regenerate_id(true) upon login.
- User input is sanitized and output is encoded to prevent XSS.
- Sensitive configuration (DB credentials, SMTP credentials) is stored outside the webroot or in a protected configuration file.

4.4 UI/UX Design

- Mobile-first responsive design implemented with Bootstrap grid and responsive utilities.
- Forms show inline validation messages and accessible labels.
- Dark/light mode toggle implemented via CSS variables and persisted in user preference (DB) or localStorage for guests.

5. Implementation

This section documents the concrete implementation choices and step-by-step procedures performed during development.

5.1 Technologies and Libraries

- Frontend: HTML5, CSS3, Bootstrap 5, JavaScript (ES6)
- Backend: PHP 8.x
- Database: MySQL 8.x
- Mail: PHPMailer (installed via Composer)
- Development Tools: Composer, Git, VS Code

5.2 Directory Structure (Suggested)

```
project-root/
├─ public/              # webroot
│  ├─ index.php
│  ├─ assets/
│  │  ├─ css/
│  │  ├─ js/
│  │
├─ src/
│  ├─ controllers/
│  ├─ models/
│  ├─ views/
│  └─ helpers/
├─ config/
│  └─ config.php
├─ vendor/              # Composer packages (PHPMailer)
├─ sql/
    └─ schema.sql
```

5.3 Registration Flow (Step-by-step)

- **Front-end form**: User completes username, email, password, confirm password fields. JavaScript performs immediate validation (required fields, email format, password strength and match).
- **Submit**: Form posts to register.php (or controller endpoint).
- **Server-side validation**: PHP validates inputs (sanitization, uniqueness checks for username/email, password strength).
- **Password hashing**: Use password_hash() to store the password hash.
- **Create user record**: Insert into users with is_active = 0.
- **Create verification token**: Generate a secure random token (e.g., bin2hex(random_bytes(32))) and insert into email_verifications with expiry (e.g., 24 hours).
- **Send verification email**: Use PHPMailer to email the verification link containing the token.
- **User confirms**: User clicks link, server verifies token and sets is_active = 1.

5.4 Login Flow (Step-by-step)

- **Front-end form**: email/username + password with optional “Remember Me” checkbox.
- **Server-side authentication**: Fetch user by email/username, verify is_active flag, then verify password with password_verify().
- **Session initialization**: Call session_regenerate_id(true), set session variables (user_id, username, logged_in_at).
- **Optional remember-me**: Implement secure persistent login tokens (set in DB and cookie) OR limit to session-based login for simplicity.
- **Record login**: Insert a row into login_history with IP address and user agent.

5.5 Profile CRUD

- Create: Profile created at registration or when user fills additional details.
- Read: Protected profile page queries user_profiles using the authenticated user_id.
- Update: Edit form validated on front-end and back-end; file uploads for avatars handled securely with checks on file type and size.
- Delete: Provide account deletion option with confirmation and optional soft-delete flag; remove or anonymize personally identifiable information as appropriate.

5.6 Password Reset Mechanism

- **Request**: User submits email in “Forgot Password” form.
- **Token creation**: Generate secure token and insert into password_resets with a short expiry (e.g., 1 hour).
- **Email token link**: PHPMailer sends a secure link to the user.
- **Reset**: User clicks link, supplies new password; server verifies token, updates password_hash and invalidates the token.

5.7 Email Verification

- Implemented with email_verifications table and PHPMailer.
- Verification link: https://example.com/verify.php?token=<token>
- Server verifies token and expiry, then activates the account.

5.8 Login History (Activity Log)

- Upon each successful login, insert user_id, login_time, ip_address, and user_agent into login_history.
- Provide an authenticated view where the user can inspect recent logins with pagination.

5.9 Dark/Light Mode Toggle

- Implement toggling using CSS variables.
- Persist user choice in users.theme_preference when logged in; fallback to localStorage for guests.
- Apply preference on page load via server-side rendering or client-side initialization.

5.10 Error Handling and Logging

- Display user-friendly error messages; log technical details in server-side logs (do not expose stack traces to users).
- Centralize logging (a helper) that writes to a secure log file with log rotation.

6. Testing

6.1 Test Plan

Testing covers functional, usability, compatibility, and security tests.

Functional tests: - Successful registration and email verification - Login with valid credentials - Rejection with invalid credentials - Password reset flows - Profile update and deletion

Usability tests: - Responsive layout on common screen sizes (mobile, tablet, desktop) - Form validations and accessibility labels

Compatibility tests: - Browser testing: Chrome, Firefox, Safari, Edge

Security tests: - Attempt SQL injection in input fields - Attempt XSS payloads in profile fields - CSRF tests for state-changing forms

6.2 Sample Test Cases

Test Case	Steps	Expected Result
Registration	Submit valid registration form	User created, verification email sent
Verification	Click verification link	Account activated
Login	Provide credentials	Session established, login recorded
Password Reset	Request reset and set new password	Password updated, token invalidated

6.3 Test Results Summary

- All primary functional cases passed during development.
- Security tests mitigated simple SQLi attempts via prepared statements; XSS prevented by output encoding.
- Responsive layout validated on standard breakpoints.

7. Deployment and Installation

7.1 Prerequisites

- Web Server: Apache or Nginx
- PHP 8.x with extensions: PDO, mbstring, openssl, fileinfo
- MySQL 8.x
- Composer (for PHPMailer and other dependencies)
- HTTPS certificate (recommended for production)

7.2 Step-by-step Installation

- Clone the repository: git clone <repo-url> project-root
- Navigate to project directory and install dependencies: composer install
- Create a MySQL database and user with appropriate privileges.
- Import sql/schema.sql using mysql -u user -p database < sql/schema.sql.
- Copy config/config.example.php to config/config.php and set DB and SMTP credentials. Ensure config.php is not publicly accessible.
- Set proper file permissions for public/assets/uploads directory.
- Configure Apache virtual host to point to project-root/public as the document root. Enable HTTPS.
- Verify php.ini settings for uploads, error reporting (production: display_errors = Off).

7.3 Running Locally

- Use built-in PHP server for quick testing: php -S localhost:8000 -t public
- For production, configure Apache/Nginx and enable HTTPS.

8. User Manual and Usage Instructions

8.1 Registration

- Visit /register.php.
- Fill in username, email, password and confirm password.
- Submit. Check email and click the verification link.

8.2 Login

- Visit /login.php and provide credentials.
- On successful login, user redirected to /dashboard.php.

8.3 Email Verification

- Click verification link in email. If the token is valid, the account becomes active.

8.4 Password Reset

- Visit /forgot-password.php, enter registered email.
- Click link sent to email and reset password within the token expiry time.

8.5 Profile Management

- Access /profile.php to view and /edit-profile.php to update profile fields or upload avatar.

- Use delete account option for permanent removal after confirmation.

8.6 Dark/Light Mode

- Toggle in UI. Preference saved to the profile and applied on subsequent logins.

8.7 Viewing Login History

- Access /login-history.php to view recent login events (time, IP, user agent).

9. Security Considerations

- **Password Storage:** Use password_hash() and password_verify().
- **SQL Injection:** Use prepared statements and parameterized queries.
- **XSS Protection:** Escape output using htmlspecialchars() before rendering user-supplied text.
- **CSRF Protection:** Generate and validate CSRF tokens for state-changing requests.
- **Session Security:** Use secure, HttpOnly cookies; regenerate session IDs after login; limit session lifetime.
- **File Uploads:** Validate MIME types and file sizes; store uploads outside of webroot or with randomized filenames.
- **Transport Security:** Enforce HTTPS in production to protect credentials and tokens in transit.

10. Project Management and Documentation

- **Version Control:** Git with meaningful commit messages.
- **Issue Tracking:** Use GitHub Issues or similar to track tasks and bugs.
- **Documentation:** Include README with setup steps, commented code, and this project report. Inline comments in code explain logic and security rationale.

11. Limitations and Future Work

11.1 Limitations

- The dummy payment simulation does not connect to a real payment gateway—only simulates payment states.
- Email delivery depends on SMTP provider reliability and appropriate configuration.
- Advanced account recovery (multi-factor authentication) is not implemented.

11.2 Future Enhancements

- Integrate a real payment gateway (Stripe, PayPal) for production payments.
- Implement OAuth (Google, Facebook) for social login.
- Add two-factor authentication (2FA) via SMS or authenticator apps.
- Expand audit logging and provide administrative dashboards.

12. Conclusion

This project demonstrates a complete, secure, and responsive web application for user registration, authentication, and profile management using PHP and MySQL. The system includes essential security measures, friendly user interfaces, email-driven workflows for verification and password reset, and additional usability features such as dark/light mode and login history. The codebase, database schema, and documentation provided will serve as a solid foundation for extension and deployment.

13. References

- *[1] The PHP Group, "PHP Manual," The PHP Group, 2024. [Online]. Available: https://www.php.net/manual/en/. [Accessed: Sep. 23, 2025].*

- [2] M. Bointon et al., "PHPMailer: The classic email sending library for PHP," GitHub, 2024. [Online]. Available: https://github.com/PHPMailer/PHPMailer. [Accessed: Sep. 23, 2025].

- [3] The Open Web Application Security Project (OWASP) Foundation, "OWASP Top 10:2021," OWASP, 2021. [Online]. Available: https://owasp.org/Top10/. [Accessed: Sep. 23, 2025].

End of Report

Source Code:
https://github.com/neelislam/PHP_for_haters/tree/main/Final_Project/userapp